

 Devinterview-io / html5-interview-questions



[Code](#) [Issues 1](#) [Pull requests](#) [Actions](#) [Projects](#) [Security](#) [Insights](#)



HTML5 interview questions and answers to help you prepare for your next technical interview in 2025.

☆ 164 stars    36 forks    2 watching    Branches    Activity

 Tags

 Public repository

 main ▾    1 Branch    0 Tags   



[Go to file](#) [Add file ▾](#) [Code ▾](#) 

 Devinterview-io 100 Essential HTML5 Interview Questions in 2025 7269a6a · 6 months ago 

 README.md   100 Essential HTML5 Interview Questions in...   6 months ago

 README  

# 100 Essential HTML5 Interview Questions in 2025

## Prepping for a Web or Mobile Dev Interview?

Check out [Devinterview.io](#) for 5325+ questions covering 58 Full-Stack development topics

[Kickstart your prep →](#)

You can also find all 100 answers here  [Devinterview.io - HTML5](#)

## 1. What does *HTML* stand for and what is its purpose?

HTML or Hyper Text Markup Language is the standard language for creating web pages and applications. HTML5, the latest version as of 2022, introduces several new elements and attributes, elevating user experience and software application standards.

HTML is responsible for structuring web content, ensuring accessibility, and guiding how web pages are visually presented. It remains the foundational structure for running nearly all web content.

### Core Functionalities

- **Structuring Content:** Tags like `<header>`, `<footer>`, and `<section>` divide content, streamlining its organization.
- **Embedding Media:** HTML provides tags to incorporate multimedia such as images, audio, and video.
- **Form Handling:** Interactive sections such as user input forms are defined with input and label tags.
- **Hyperlinks:** Essential for navigation, hypertext links like `<a>` anchor content within or outside the webpage.
- **Accessibility Features:** Semantic tags like `<nav>` and `<article>` not only structure data but also improve accessibility for users relying on screen readers.
- **Integration of Other Technologies:** Can integrate with scripting languages like JavaScript and libraries and frameworks like Bootstrap for enhanced visual appeal.

### Compatibility and Development

The primary objective of HTML5 is to improve the language's support for the latest multimedia, while keeping it easily readable by humans. It was designed to be backward and forward compatible, so that content written in previous versions can be seamlessly integrated and interact with content authored in subsequent versions.

## Visual Presentation and User Interface Adaptations

HTML5 is engineered to provide more flexibility, control, and aesthetic maturity for web pages and web-based software applications. Web developers can use it to craft modern web interfaces with rich visual and multimedia experiences. It also allows for more responsive and adaptive design, ensuring optimal viewing on a variety of devices and screen sizes. This reflects a broader shift in technology toward a more device-agnostic user experience.

## The Role of CSS and JavaScript

While HTML offers static content, **CSS** and **JavaScript** enable additional layers of styling, interactivity, and dynamic content updates. The integration of these three technologies (referred to as **HTML-CSS-JS**) stands as the trio that forms the backbone of almost all web-based content. They're often presented as **HTML5-CSS3-JS** to signify unified modern best practices. Online, there's even tools that combine these technologies into a single framework or language such as WebAssembly or Dart. The trio represents a more modular approach, allowing distinct teams to focus on individual layers, streamlining development in larger projects. Mastering their intersection helps in designing a robust and cohesive user experience. This concept is captured by the acronym "PEA", which stands for the Platform (HTML), the Engine (JavaScript), and the Appearance (CSS). Each category focuses on a distinct aspect of user experience.

## Practical Uses

- **Website Development:** All traditional web resources, from simple blogs to expansive e-commerce sites, remain based primarily on HTML.
- **Web Applications:** Web technologies have evolved significantly, allowing for sophisticated applications like Google Docs, Trello, and Slack to run entirely in a web browser. HTML5 has played a pivotal role in this development.
- **Advertising & Media:** HTML5's advanced media handling tools have made it the standard for online ads and multimedia content.

## 2. Describe the basic structure of an *HTML document*.

HyperText Markup Language (HTML) serves as the backbone of web content, defining its structure and semantics. We will now walk you through the fundamental elements of an HTML document.

### Basic Structure of an HTML Document

An HTML document consists of two primary sections: the `head` and the `body`.

#### Document Type Declaration (DOCTYPE)

The **Document Type Declaration** (DOCTYPE) is not an HTML tag; it's an instruction to the web browser about what version of HTML the page is written in.

```
<!DOCTYPE html>
```



This declaration shows that the document is an HTML5 document.

#### HTML Element

The `html` element is the root element of an HTML page. It encompasses the entire content, both head and body.

```
<html>
  <!-- Head and Body Sections Are Nested Inside -->
</html>
```



#### Head Section

The `head` section provides meta-information about the document. It isn't displayed in the web browser itself but serves various other purposes, from providing a title to linking external resources.

```
<head>
  <!-- Title and Meta-Tags, Styles, Scripts, etc. -->
</head>
```



#### Title Element

The `title` element specifies the document's title, which is displayed in the browser's title bar or tab.

```
<title>Your Page Title</title>
```



#### Body Section

The `body` section encapsulates the document's visible content—what users see and interact with.

```
<body>
  <!-- Content Visible to Users: Headings, Paragraphs, Images, etc. -->
</body>
```



## 3. What do *DOCTYPE* and *html lang* attributes do?

Document Type (DOCTYPE) and the `lang` attribute play crucial roles in our webpages.

### DOCTYPE: Defining Document Type and Validation Mode

#### Purpose

- Specifies the HTML or XHTML version used in the document.
- Identifies parsing method and algorithm for the web browser, affecting consistency.

#### Code Example

The `<!DOCTYPE>` declaration is placed at the very top of the HTML file, even before the `<html>` tag begins.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Page Title</title>
  </head>
  <body>
    <!-- Content -->
  </body>
</html>
```



### Lang Attribute: Language Specification

The `lang` attribute, present in the HTML tag, specifies the primary language used in the document. Its value is a primary language subtag as defined in RFC 5646 (BCP 47) and it can include a valid language code, a valid language code followed by a valid region code, or simply "und" for unspecified language.

#### Code Example

```
<!DOCTYPE html>
<html lang="en-US">
  <head>
    <title>Page Title</title>
  </head>
  <body>
    <h1>Welcome</h1>
```



```
<p>This is a demo page.</p>
</body>
</html>
```

## 4. What is the difference between *head* and *body* tags?

While the `<head>` and `<body>` tags are fundamental to every HTML document, they serve distinct purposes and are located in separate areas of the web page.

### Key Distinctions

#### 1. Role and Content

- **Head:** Houses meta-information, such as document title, character encoding, and stylesheets, all of which are essential for page setup but not visible to the user.
- **Body:** Contains the bulk of visible content, including text, images, videos, links, and more.

#### 2. Placement in the HTML File

- **Head:** Precedes the body and provides setup before actual content is rendered.
- **Body:** Follows the head section and encompasses all visible content.

#### 3. Common Elements in Each Section

- **Head:** Typically links to CSS files or may have inline CSS, contains the document title, any JavaScript reference, character set declaration, and meta tags.
- **Body:** Holds structural components like headers, navbars, articles, sections, and the footer, along with visual content like images and visible text.

### Visual Representation in the HTML File

- **`<head>` Section:**

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <link rel="stylesheet" type="text/css" href="styles.css">
  </head>
  <body>
    <!-- Content Here -->
  </body>
</html>
```

- **`<body>` Section:**

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <link rel="stylesheet" type="text/css" href="styles.css">
  </head>
  <body>
    <header>
      <h1>Welcome!</h1>
      <nav>
        <ul>
          <li><a href="#">Home</a></li>
          <li><a href="#">About</a></li>
          <li><a href="#">Contact</a></li>
        </ul>
      </nav>
    </header>
```

```
<section>
  <h2>Recent Posts</h2>
  <article>
    <h3>Post Title</h3>
    <p>Post content goes here.</p>
  </article>
</section>

<footer>
  <p>&copy; 2023 MySite</p>
</footer>

</body>
</html>
```

## 5. Can you explain the purpose of *meta tags* in HTML?

**Meta tags** provide metadata about a web page through information invisible to visitors but essential for search engines, social media, and other web technology. This metadata includes details such as the page's title, keywords, and description.

### Key Meta Tags

- **Meta Description:** A concise summary of the page's content, often used in search engine results.
- **Meta Keywords:** Historically used to specify relevant keywords for the page, but they have been largely deprecated due to abuse by spammers.
- **Meta Robots:** Directs search engine bots on how to interact with the page, such as index it for search results, follow its links, or refrain from both.
- **Meta Viewport:** Crucial for responsive design, it guides the browser on how to scale and display the page, especially useful for mobile devices.
- **Meta Charset:** Defines the character encoding used on the webpage, ensuring text is displayed correctly.
- **Meta Author:** Identifies the page's creator or author.
- **Open Graph, Twitter Cards:** Specialized meta tags used by social platforms like Facebook and Twitter to optimize page sharing.
- **Canonical URL:** Indicates the preferred URL when a page can be accessed through multiple paths.
- **Refresh and Redirect:** Older, less common meta tags that dictate page behavior.

### Code Example: Common Meta Tags

Here is the HTML code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="description" content="This is a sample web page with a concise description.">
  <meta name="keywords" content="HTML, meta tags, web design, SEO">
  <meta name="author" content="John Doe">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sample Web Page</title>
</head>
<body>
  <!-- Page content goes here -->
</body>
</html>
```



### Responsible Use of Meta Tags

With search engines evolving, many tags have diminished in significance. Here's the current state:

- **Still Relevant:** Meta Description, Viewport, Charset, Author, and Canonical
- **Limited Effect:** Keywords, Refresh, and Robots
- **Specialized Fields:** Open Graph, Twitter Cards are necessary for tailored content on social platforms

To maintain a robust online presence, focus on high-quality content, user experience, and technical soundness, and don't solely rely on meta tags.

## 6. How do you link a *CSS file* to an *HTML document*?

Linking a CSS file to an HTML document is a fundamental step for styling. This is generally done by indicating the CSS file's path in the `head` section of the HTML file using `<link>` tags.

### HTML Link Tag: `<link>`

HTML uses the `<link>` tag to **integrate external resources** such as CSS files.

#### Syntax

```
<link rel="stylesheet" href="path/to/style.css">
```



- **rel:** Specifies the type of relationship between the current document and the linked file. For CSS, it should be set to "stylesheet".
- **href:** Points to the location of the external CSS file. This can be via an **absolute URL** (i.e., `http://...`) or a **relative path** to the HTML file.
- **type:** Supplied for legacy purposes but is not required given the file is a CSS file.

#### Code Example: Using the Link Tag

Here is the HTML code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <link rel="stylesheet" href="path/to/style.css">
</head>
<body>
  <!-- Body content -->
</body>
</html>
```



## 7. How do you link a *JavaScript file* to an *HTML document*?

To link a JavaScript file to an HTML document, you need to use the `<script>` HTML tag. There are two primary ways to do this:

1. **External Script File:** Link a separate JavaScript file to your HTML document.
2. **Inline Script:** Embed JavaScript code directly within your HTML file.

### External Script File

To use an external JavaScript file, follow these steps:

1. **Create the JavaScript File:** Save your JavaScript code in a separate file with a `.js` extension. For example, `script.js`.
2. **Link the JavaScript File to your HTML Document:** Add the following code within the `<head>` or at the end of the `<body>` section of your HTML file.

```
<script src="path-to-your-js-file.js"></script>
```



Replace `path-to-your-js-file.js` with the actual path to your JavaScript file.

### Best Practices

- **Placement:** It's good practice to place your `<script>` tags at the end of the `<body>` section, just before the closing `</body>` tag. This ensures that the HTML content loads first, which can improve the website's initial rendering speed.
- **Syntax:** The HTML5 specification does not require a closing tag for the `<script>` element.

### Inline Script

You can also include JavaScript directly within your HTML file. This is called an "inline script." To do this, encase your JavaScript code within `<script>` tags, like this:

```
<script>
  // Your JavaScript code goes here
</script>
```



### Best Practices

- **Content Separation:** For better code organization, it's often better to keep your JavaScript in a separate file, especially for larger applications.
- **Caching:** When using an external JavaScript file, the browser caches the script, which can speed up your site on subsequent visits. However, if the script changes often, this caching can be a problem.
- **Maintainability and Reusability:** Utilizing an external JavaScript file allows for better code management, reusability, and ease of making updates or fixes across multiple HTML files.

### Example HTML File

Here is the code:

### Implementation: HTML File

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script src="path-to-your-js-file.js"></script>
</head>
<body>
  <!-- Your content here -->
  <script>
    // Inline JavaScript code here.
  </script>
</body>
</html>
```



## 8. How do you add a *comment* in HTML and why would you use them?

To add a **comment** in HTML, wrap it between `<!--` and `-->`.

For example:

```
<!-- This is a comment -->
<p>Hello, World!</p>
```



### Role of Comments in Development

Comments ensure clear code comprehension and can be used for:

- **Instructions:** Guiding developers on next steps.
- **Documentation:** Articulating intricate code segments.
- **Debugging:** Temporarily removing portions for bug testing.
- **Reminders:** Highlighting sections for later revision.

## Best Practices for Using Comments

- **Purposeful Clarity:** Comments must explain what the code does, not how. Code and inline comments should clarify how the code works.
- **Relevance:** Avoid stating the obvious and focus on unique or complex components.
- **Conciseness:** Keep comments brief to reduce visual clutter.
- **Regular Maintenance:** Update or remove outdated comments to maintain accuracy.

## When are Comments Unnecessary?

- **Trivial Cases:** Comments like "wrapper" or "tag" denote the obvious.
- **Self-Explanatory Code:** Writing **self-descriptive** code eliminates the need for specific comments.

## 9. How do you serve your page in multiple *languages*?

Let's discuss the **best practices** for serving web pages in multiple languages and the corresponding HTML5 tag, `<html lang="en">`.

### Language Tag

For serving content in multiple languages and optimizing accessibility and search engine performance, you should use the `lang` attribute on the `<html>` tag. This is considered a **best practice**, even if the page is only in English.

```
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Your Website</title>
</head>
<body>
  <!-- Page content here -->
</body>
</html>
```



### Common Language Codes

Most languages follow the two-letter ISO 639-1 code, such as "en" for English or "es" for Spanish. Some languages also use an extended ISO 639-2 or 639-3 code, which might require three to four letters, like "por" for Portuguese.

For dialects or region-specific content, you can use a hyphen, followed by an ISO 3166-1 alpha-2 country code. For instance:

- "en-GB" for British English
- "es-ES" for Spanish as spoken in Spain
- "pt-BR" for Brazilian Portuguese
- "pt-PT" for European Portuguese

Understand that while the `lang` attribute assists in accessibility, user agents may not always recognize or act upon these subtags.

### SEO Considerations



Serving content in multiple languages comes with SEO responsibilities. One common practice is to assign a *language-specific* URL for each version of your content. In addition to this, utilize human-readable URLs to effectively communicate the language and the content topic/design.

For instance, use:

- `example.com/en-US/about` for pages in American English.
- `example.com/es-MX/sobre` for those in Mexican Spanish.

## AI-Clearance Required

This technique requires further validation and clearance upon implementation as a lot is dependent on SEO constraints and localized content.

## 10. What are *data-\** attributes and when should they be used?

**Data attributes** in HTML5, often referred to as **data-\* attributes**, help embed custom data within HTML elements. This presents a powerful tool for web developers, facilitating streamlined **JavaScript** and **CSS** operations.

### Core Benefits

- **Accessibility:** Data attributes are easily accessible through the `dataset` API in JavaScript.
- **Ignoring formatting tactics:** In places where content served by backend, cannot assume the content to always be JSON encoded, shortened, or have odd formatting.
- **Data Isolation:** For better maintenance of web documents. Data attributes have clear, defined roles within HTML.

### Code Example: Using Data Attributes

Here is the HTML & JavaScript:

```
<div id="user" data-name="John Doe" data-age="25"></div>

<script>
  const userDiv = document.getElementById('user');
  console.log(userDiv.dataset.name); // Output: "John Doe"
  console.log(userDiv.dataset.age);  // Output: "25"
</script>
```



### Appropriate Use-Cases

- **Custom Content for DOM Elements:** For attaching extra information or configuration settings exclusively relevant to an HTML element.  
  
Example: A `div` may have a `data-show-tooltip` attribute set to `true` to indicate it should display a tooltip.
- **Interactivity Configuration:** When working with user-made widgets, data attributes can specify how they behave in a more structured, intended manner. Useful in contexts where individual DIV or section blocks have interactivity toggles, or categories.
- **E-Commerce & Web Products:** To store product-specific IDs or additional details as they pertain to the DOM representation of a product in a catalog.
- **Styling Signifiers:** You can leverage data attributes in CSS for different types of styling like category colors, hover effects, or even in JavaScript-based CSS declarations.

## 11. What is the difference between *b* and *strong* tags?

The `<b>` and `<strong>` tags are **both used for text emphasis** in HTML, but they have different semantic meanings.

### Bold vs. Strong

- The purpose of the `<b>` tag is to make the text **bold**, mainly for visual styling.
- The `<strong>` tag, on the other hand, **semantically emphasizes** the text, indicating its importance.

## Semantic Importance

The use of semantic tags like `<strong>` is beneficial for components like screen readers, browsers, and search engines, which can provide better user experience or understanding of content with proper emphasis.

### Code Example: B vs. Strong

In the HTML, the content "Caution" is visually bold and the content "Urgent Notice!" is both visually bold and semantically strong.

```
<p>
  <b>Caution</b>: This action cannot be undone.
  <br>
  <strong>Urgent Notice!</strong> Please save your work before proceeding.
</p>
```



## General Best Practice

- **Visual Styling** is usually left to CSS. `<b>` should be used with caution, if at all, as it becomes redundant in many scenarios due to CSS's wide adoption.
- **Semantic Tags** like `<strong>` provide context, clarity, and accessibility to the content.

## 12. When would you use *em* over *i*, and vice versa?

Let's see the difference between 'em' and 'i' HTML tags.

### When to Use 'em'

The 'em' tag italicizes the text by default and should be reserved for occasions when emphasis is needed.

One potential usage could be for interactive instructions:

```
<p><strong>Press</strong> <em>Enter</em> to submit.</p>
```



### When to Use 'i'

The 'i' tag, or *italics* tag, is often avoided for text styling. Instead, consider semantic HTML, CSS, or more explicit HTML options like `<em>` for emphasis, when possible.

Here's an example of `<em>` combined with CSS for an additional bit of fluorescence.

```
<p>His <em style="background-color: yellow; color: red;">anger</em> was palpable.</p>
```



## 13. What is the purpose of *small*, *s*, and *mark* tags?

The `small`, `s`, and `mark` HTML5 tags are used to **alter the structure and presentation** of text content.

### `<small>`

The `<small>` tag indicates that the enclosed text is of lesser importance, typically used for **fine print**, legal disclaimers, copyright notices, etc.

Here are examples:

### Use Case

```
<footer>
  <small>&copy; 2022 Company Name</small>
</footer>
```

### <s>

The `<s>` tag, which stands for "strike," is a non-semantic, obscure tag that is often replaced with a more meaningful tag, such as `<del>` for "deleted" content. However, it still visually **strikes out** its content.

#### Use Case

```
<p>Your discount code is: <s>EXPIRED123</s></p>
```

#### Visual Representation

Your discount code is: ~~EXPIRED123~~

### <mark>

The `<mark>` tag is used to **highlight** or **set apart** text without specifying any additional semantic information.

#### Use Case

```
<p>Important: Please <mark>schedule your appointment</mark> at least 48 hours in advance.</p>
```

## 14. What are *semantic HTML tags* and why are they important?

**Semantic HTML tags** provide both structure and meaning to web content. They allow crawlers, browsers, and even assistive technologies to understand content better and present it more effectively. This approach improves accessibility and search engine optimization, making pages easier to maintain and understand.

### Benefits of Semantic Tags

- **SEO and Accessibility:** Employing semantic tags improves your page's search engine ranking and ensures it's accessible to all users, including those with disabilities.
- **Consistent Structure:** Semantic tags establish a cohesive layout, vital for large websites or platforms.
- **Relevance to Bots and Crawlers:** Search engine algorithms dissect web pages more accurately when content is correctly labeled.
- **Content Division:** Segregating content by their meaning makes the document more understandable and maintainable.

### Common Semantic Tags

- `<p>` : A paragraph.
- `<h1>` - `<h6>` : Headings, with 1 (highest) to 6 (lowest) levels.
- `<ul>` / `<ol>` : Unordered or ordered list.
- `<li>` : List item inside a list.
- `<a>` : Anchor, used for links.
- `<img>` : An image.
- `<figure>` / `<figcaption>` : For a figure such as an image, with accompanying caption.

### Necessary vs. Optional Tags

While essential tags like `<header>` and `<footer>` indicate crucial sections, many are optional based on the website's nature or the page's segregation needs. For example, a blog may require the `<article>` tag, while a retail site might not.

In modern web development, the clear distinction offered by semantic tags is invaluable for quick comprehension and maintenance, yielding superior results for both users and developers.

## Code Example: Before vs. After Semantic HTML

Consider the before and after examples to see the impact of semantic tags.

### Before Semantic HTML

```
<div class="nav">
  <div class="logo">
    <a href="#">Logo</a>
  </div>
  <div class="nav-links">
    <a href="#">Home</a>
    <a href="#">About</a>
    <a href="#">Contact</a>
  </div>
</div>
<div class="main-wrapper">
  <div class="image">
    
  </div>
  <div class="content">
    <h3>Welcome</h3>
    <p>Some welcome text here.</p>
  </div>
</div>
<div class="footer">
  <p>© 2022 Company Name</p>
</div>
```



### After Implementing Semantic Tags

```
<header>
  <div class="logo">
    <a href="#">Logo</a>
  </div>
  <nav>
    <a href="#">Home</a>
    <a href="#">About</a>
    <a href="#">Contact</a>
  </nav>
</header>

<main>
  <figure>
    
    <figcaption>A beautiful landscape</figcaption>
  </figure>
  <section>
    <h1>Welcome</h1>
    <p>Some welcome text here.</p>
  </section>
</main>

<footer>
  <p>© 2022 Company Name</p>
</footer>
```



## 15. How do you create a *paragraph* or a *line break* in HTML?

In HTML, to create a **paragraph**, use `<p>...</p>` tags, and to insert a **line break**, use `<br>` tag.

### Paragraphs in HTML

Traditional paragraph formatting in HTML is achieved using the `<p>` tag. The browser's default styling generally adds **spacing** to the top and bottom of each `<p>` element, creating distinct paragraphs.

### Syntax

```
<p>
  This is an example of a paragraph. The text enclosed within the &lt;p&gt; tags represents a single paragraph.
</p>
```

### Visual Representation

This is an example of a paragraph. The text enclosed within the `<p>` tags represents a single paragraph.

## Line Breaks in HTML

To insert a simple line break in an HTML document, use the `<br>` tag. This tag doesn't require a closing equivalent.

### Syntax

```
First Line<br>Second Line
```

### Visual Representation

First Line

Second Line (This text doesn't render the line break; it's just to show the raw HTML.)

## Multi-line Text Elements

In HTML, the `<textarea>` tag allows the input of several lines of text. Nonetheless, it does not auto-format for paragraphs. It **wraps text** instead, and vertical scroll bars might be enabled, based on the template and content.

### Syntax

```
<textarea rows="4" cols="50">
  This is a multi-line text area.
  It doesn't automatically create separate paragraphs.
  Text wraps based on dimensions supplied.
</textarea>
```

### Visual Representation

```
<textarea rows="4" cols="50">
  This is a multi-line text area.
  It doesn't automatically create separate paragraphs.
  Text wraps based on dimensions supplied.
</textarea>
```

Explore all 100 answers here 🙌 [Devinterview.io - HTML5](https://devinterview.io/html5-interview-questions)

## Releases

No releases published

## Packages

